

Apprentissage par renforcement

2025-2026

Attention : Ce TP ne fonctionne pas dans un environnement de type notebook. Il a été testé uniquement avec un environnement Spyder et Vscode

La partie 2 nécessite l'installation de Pytorch

Le sujet comporte 2 parties, la première partie est dédiée à l'apprentissage par renforcement à l'aide de l'algorithme Q-learning tandis que la seconde partie est consacrée à l'apprentissage par Deep Q Networks

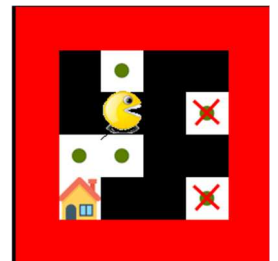
Il existe une multitude d'article et de post sur le sujet. Voir par exemple : <https://www.mlq.ai/deep-reinforcement-learning-q-learning/>

Partie 1 : jeu de pacman simplifié

Il s'agit d'apprendre la stratégie du Pacman par Q-learning

Le jeu

Dans sa version minimale, le jeu est composé d'un damier dans laquelle le pacman peut évoluer. Certaines cases peuvent contenir de la nourriture (point vert) tandis que d'autres cases peuvent contenir du poison (croix rouge). La pacman doit se rendre à sa maison en ayant mangé la nourriture (s'il y en a) et en évitant les cases poisons (s'il y en a).



Lors de chaque déplacement le pacman perd des points de récompense, il perd également des points de récompense s'il passe sur une case poison. En revanche il gagne des points s'il mange de la nourriture. Il doit donc manger toute la nourriture avec un minimum de déplacement afin de maximiser la somme de ses points de vie (cumulated reward).

La description de la configuration du jeu est contenue dans les fichiers (boardXXX.txt), l'annexe 1 fournit le format exact. Les fichiers boardX.txt sont stockés dans le dossier « boards »

Les actions possibles

Dans la version qui vous est fournie, le pacman ne peut effectuer que les 4 actions suivantes : descendre (action n°0), monter (action n°2) , aller à droite (action n°3), aller à gauche (action n°1).

Les récompenses (rewards)

Elles sont définies à la création du jeu. Les valeurs fournis ici ne sont pas forcément optimales !

```
env = Pacman(file,STEP_REWARD = -0.1,FOOD_REWARD=10, WIN_REWARD = 10, POISON_REWARD = -50)
```

STEP_REWARD : pénalité pour un déplacement d'une case

WIN_REWARD : récompense lorsque la tortue est à sa maison et a mangé toute la nourriture

POISON_REWARD : pénalité lorsque pacman passe sur une case poison

FOOD_REWARD : récompense lorsque pacman passe sur une case nourriture

L'environnement

Le programme permettant de représenter l'environnement est fourni dans le fichier PacmanenvdiscreteSpace.py. Le programme main-QL.py est le programme principal. Ce dernier permet de choisir parmi 4 modes de fonctionnement : manuel/train/play/stat :

- Le mode manuel permet de faire évoluer la tortue manuellement à l'aide des flèches et affiche des informations utiles (etat, reward, ...).
- Le mode train permet d'estimer la table Q. En fin d'apprentissage on affiche le reward cumulé en fonction des épisodes ainsi l'histogramme des états visités. Le mode stat est également exécuté en fin d'apprentissage afin de calculer des statistiques sur 100 parties
- Le mode play permet à l'agent de jouer une partie à l'aide de la table Q précédemment apprise en initialisant le pacman à une position aléatoire.
- Le mode stat permet à partir d'une table Q préalablement apprise de tester les performances sur 100 parties.

La ligne de code ci-dessous dans le programme principal permet de créer un environnement minimaliste à partir du fichier texte (boardXX.txt) de configuration du jeu (voir annexe)

```
file = 'boardXXX'
env = Pacman(file, STEP_REWARD = -0.1, FOOD_REWARD=10, WIN_REWARD = 10, POISON_REWARD = -50)
```

A. Pour commencer

Codage de l'état : Ici l'état est codé par une valeur scalaire. L'état du système dépend de la position de pacman mais aussi de l'état des nourritures (mangée ou non). Dans ce jeu, le poison n'entre pas dans la description de l'état car il ne disparaît pas lorsque le pacman en mange. On note F_i la variable binaire associée à la nourriture numéro $i \in [0, n-1]$ tel de $F_i = 0$ lorsque la nourriture est intacte et $F_i = 1$ lorsqu'elle a été mangée. On note par la variable Pos la position de la tortue (Pos = 0 en bas à gauche). Le calcul de l'état est obtenu ici par la formule suivante :

$$State = Pos + NbreTotalPos \times \sum_{i=0}^{n-1} F_i \times 2^i$$

Expliquer en quoi ce codage permet de représenter l'état de l'ensemble du jeu. Lancer le programme principal en utilisant le fichier board0.txt qui permet de réaliser un damier minimaliste de 2x2 cases et 1 nourriture. Déplacer la tortue en mode manuel afin de balayer tous les états possibles (il faut relancer plusieurs fois le programme). D'après la formule ci-dessus, combien d'état sont-ils possibles ? Combien en avez-vous obtenu en pratique ? Justifier en dessinant la chaîne de Markov associée. Conclure.

B. Apprentissage de la table action-value function (Q)

On utilise pour cela l'algorithme Q-learning. Il est fortement recommandé de reprendre le cours et de consulter le lien suivant pour sa mise en œuvre : <https://medium.com/analytics-vidhya/reinforcement-learning-using-q-learning-to-drive-a-taxi-5720f7cf38df>

Seules 3 méthodes de l'environnement sont utiles :

```
- env.reset()                # permet d'initialiser l'environnement et positionne le pacman aléatoirement
- new_state, reward, done, info = env.step(action)    # déplace le pacman avec l'action fournie en argument
- action = env.sample()      # permet de tirer au hasard une action
```

La méthode env.step retourne le nouveau état suite à l'action « action » ainsi que le reward associé et la variable booléenne done (qui vaut 1 si le jeu est fini (le pacman est arrivé à sa maison et a mangé toutes les nourritures), sinon 0).

1. Le fichier Qlearning.py contient le code (incomplet) de la fonction Qlearning de l'algorithme de Q-learning. Il vous est nécessaire de le comprendre, puis de le compléter en vous basant sur le cours et à l'aide du lien ci-dessus. Cette fonction prend en argument l'environnement (variable env), le nombre d'actions possible (4) et le nombre d'états possibles. Elle prend également en argument les hyperparamètres (learning rate, discount factor, epsilon, ...). Elle retourne la table Q estimée.
2. Utiliser le fichier board1 pour créer un damier de 3x3 avec 1 nourriture et 1 poison. Quel est le nombre d'états ? La dimension de la Q-table ? Lancer l'apprentissage pour estimer la table Q. Commenter les

statistiques obtenues ainsi que la courbe d'apprentissage. Pourquoi certains états sont-ils plus visités que d'autres ?

Remarque : A l'aide de l'option 'play' vous pouvez lancer une partie et vérifier que le Pacman se déplace correctement.

3. Prendre $\gamma = 1$ et refaire un apprentissage. Afficher la matrice Q et Interpréter ses valeurs. Obtient t'on exactement les valeurs attendues ? Refaire un apprentissage avec $\gamma = 0,1$. Pourquoi obtient une matrice Q différente ? Cela a-t-il de l'importance ?
4. Prendre $\gamma = 0$ et refaire un apprentissage. Commenter la courbe d'apprentissage et discuter des résultats obtenus.
5. Changer les valeurs des rewards et analyser leur influence
6. On complexifie la tâche (prendre la board2). Quel est le nombre d'états ?
7. Expliquer le rôle du paramètre `exploration_decreasing_decay`. Analyser la courbe d'apprentissage pour différentes valeurs de ce paramètre. A votre avis pourquoi ici la table est bien apprise pour une large gamme de valeurs de ce paramètre ?
8. Etudier l'influence de la valeur des rewards. Que faut il faire afin que le pacman ne s'empoisonne pas ?
9. Bonus : avec les paramètres trouvés précédemment peut on apprendre correctement le jeu en utilisant la board3 ?
10. Si on change l'emplacement de la maison, est-il nécessaire de refaire un apprentissage ? Justifier

Partie 2 : Deep Q Networks (DQN)

Le jeu est identique mais on utilise un réseau de neurones à la place de la table Q. L'état du système n'est plus représenté par un scalaire mais par un vecteur d'état. Dans la version fournie le vecteur d'état est de dimension $2 + \text{nombre de nourriture}$: $[X, Y, F_1, F_2, \dots]$. Les F_i sont des flags (0 ou 1) qui mémorisent si une nourriture a été mangée ou non. Les coordonnées X et Y sont normalisées entre -1 et 1. L'ensemble de l'environnement est codé dans le fichier `PacmanEnvVectorSpace.py`

Le réseau de neurones : Dans la version de base, il s'agit d'un petit réseau du type « full connected » à 2 couches cachées de 16 neurones chacune (défini au début du fichier `DQNPtorch.py`). Pour apprendre ce réseau, vous allez mettre en œuvre un apprentissage par renforcement du type Deep Q Networks (DQN). Le code du réseau de neurone est stocké en début du fichier `DQNPtorch.py` (codé en Pytorch). Il est conseillé dans un premier temps de lire :

<https://towardsdatascience.com/reinforcement-learning-explained-visually-part-5-deep-q-networks-step-by-step-5a5317197f4b>

<https://builtin.com/artificial-intelligence/deep-q-learning>

<https://www.mlq.ai/deep-reinforcement-learning-q-learning/>

Prise en main

Vous devez avoir installé au préalable Pytorch (une carte GPU n'est pas indispensable). Le code (version basique) permettant de mettre en œuvre l'apprentissage par Deep Q-learning est également fourni (`DQNPtorch.py`). Le programme principal est dans le fichier `main-DQN-Pytorch.py`

Dans un premier temps vous pouvez explorer le fonctionnement de l'environnement de façon manuelle à l'aide des flèches du clavier. Comme dans la partie 1, lancer le programme avec la board1 et choisir l'option « manuel ».

Cette option permet d'itérer à l'infini en exécutant la ligne de commande :

```
state, reward, done, info=env.step(action)
```

Déplacer le pacman à l'aide des flèches et observer l'évolution du vecteur d'état, des valeurs du reward, du reward cumulé.

Mode apprentissage : (option train)

Les principaux hyperparamètres définis en début du fichier DQNPYtorch) sont les suivants :

- Nombre de neurones par couche (FC1_DIMS, FC2_DIMS)
- Learning rate (lr)
- exploration_decreasing_decay (vitesse de décroissance de epsilon)
- MEM_SIZE (taille du replay buffer)
- BATCH_SIZE : taille du batch
- EXPLORATION_MIN (probabilité min d'exploration)
- MAX_STEP (nombre max de pas de temps dans un episode)
- TARGET_UPDATE_FREQ (frequence de mise à jour du réseau target)
- FC1_DIMS, FC2_DIMS (taille des couches cachées du réseau deneurones)
- gamma (pondération cumulated reward)

Mode play

Le mode play utilise le réseau précédemment appris pour déplacer le pacman à partir d'une position initiale aléatoire. Les lignes de code ci-dessous permettent respectivement de fournir au réseau de neurones l'état du jeu afin que celui-ci fournisse l'action à faire (ligne 1). La ligne 2 permet d'exécuter l'action déduite par le réseau. Ces 2 lignes étant placées dans une boucle le jeu évolue jusqu'à ce qu'il se termine.

:

```
action=net.play1step(env.state())  
state, reward, fini, info=env.step(action)
```

Le programme itère jusqu'à ce que le jeu se termine (ou s'arrête au bout de 50 itérations si votre réseau est mal appris !).

Mode Stat

Ce mode est lancé automatiquement après un apprentissage ou si on choisit le mode « stat ». Ce mode lance 100 fois le jeu et compte le nombre de fois où la partie est gagnée ainsi le nombre de fois où la pacman est passé sur une case poison. Ce mode permet donc d'évaluer les performances du modèle appris.

Travail demandé :

- a. Analyser le fichier DQNPYtorch :

- Expliquer le rôle du ReplayBuffer
 - Expliquer le rôle du réseau target
- b. Il s'agit d'étudier l'influence des hyperparamètres (nombre de neurones du réseaux, paramètres d'apprentissage). Commencer par la board 1. Dans le cas d'un MLP(3,3) tel qu'il est défini dans le fichier fourni quel est le nombre de poids du réseau ? Comparer cette valeur avec la taille de la Q-table dans le cas d'un codage avec une valeur scalaire. Conclure.
- c. Lancer un apprentissage. L'apprentissage est-il correct ? A votre avis, si on change la position d'un poison, est-il nécessaire de ré-apprendre le réseau ? Même question pour la position de la maison. Dans la négative que conviendrait-il de faire ?
- d. On utilise maintenant la board2. Est-il possible d'apprendre correctement avec le MLP (3,3) ? Dans la négative modifier la taille (par exemple 8x8). Utiliser le mode play et identifier une raison pour laquelle le Pacman ne parvient pas à terminer le jeu.
- e. Utiliser une architecture 16x16 et étudier l'influence des paramètres gamma (prendre 0.5 par exemple), MEM_SIZE (prendre des valeurs << 4096), MAX_STEP (prendre des valeurs < 20). Dans tous les cas, proposer une justification.
- f. On complexifie le jeu en reprenant la board3. Le réseau 16x16 est-il de taille suffisante ? Dans la négative apporter les corrections nécessaires.
- g. En pratique l'algorithme Deep Q Network peut être instable et peut poser des problèmes de convergence. On lui préfère une version améliorée de type Double Deep Q Network (voir <https://towardsdatascience.com/double-deep-q-networks-905dd8325412>). Modifier la méthode learn de la classe DQN_solver afin de l'adapter à l'algorithme Double DQN. Tenter d'apprendre le jeu avec la board3.

Annexe 1

Fichiers de configuration (boardXX.txt)

Ligne 1 : taille de l'aire de jeu

Ligne 2 : 1 1 H # Position de la maison

Lignes suivantes : X Y Value

X : abscisse

Y : ordonnée

Value :

F : Food

P : Poison

La numérotation des coordonnées X et Y commence à la valeur 1. Ainsi la case (1,1) est en bas à gauche